

Algoritmi - Domača naloga 1

Luka Šveigl, ls6727@student.uni-lj.si, 63200301

March 2025

1 Časovna zahtevnost

1.1 1. naloga

```
funkcija t1(n: int) -> int {  
    s: int = 0  
    for i: int = 0; i < n; i++ {  
        s += i + 1  
    }  
    vrni s  
}
```

Ker funkcija sestoji iz preproste zanke, ki se izvede n -krat, je časovna zahtevnost algoritma $O(n)$.

1.2 2. naloga

```
funkcija t2(n: int) -> int {  
    s: int = 0  
    for i: int = 0; i < n; i++ {  
        for j: int = 1; j < i; j *= 2 {  
            s += t1(i) // t1 je funkcija iz prejšnje naloge  
        }  
    }  
    vrni s  
}
```

Klic funkcije $t1$ ima časovno zahtevnost $O(n)$. Ker se zunanja zanka izvede n -krat, in notranja zanka $\log n$ -krat, ima funkcija $t2$ časovno zahtevnost $O(n^2 \log n)$.

1.3 3. naloga

```
funkcija cantor(n: int, a: float, d: float) {  
    if n < 1 {  
        return d - a  
    }
```

```

}
b: float = 2 * a / 3 + d / 3
c: float = a / 3 + 2 * d / 3
vrni cantor(n / 2, a, b) + cantor(n / 2, c, d)
}

```

Ker je funkcija `cantor` rekurzivna, uporabimo Mojstrov teorem. Po Mojstrovem teoremu, imamo v našem primeru vrednosti $a = 2$, $b = 2$ in $d = 0$, kar spada v prvi primer Mojstrovega teorema:

$$T(n) = O(n^{\log_2 2}) = O(n)$$

Torej je časovna zahtevnost tega algoritma $O(n)$.

2 Rodovne funkcije

2.1 4. naloga

Andrej ima na voljo kovance v vrednosti 1, 2, 5 in 10c, torej lahko njegov problem zapišemo z naslednjo rodovno funkcijo:

$$G(x) = \frac{1}{1-x} \times \frac{1}{1-x^2} \times \frac{1}{1-x^5} \times \frac{1}{1-x^{10}}$$

Če izračunamo to funkcijo, dobimo rezultat 16 različnih kombinacij.

Petra ima na voljo kovance v vrednosti 1 in 2 centa, torej lahko njen problem zapišemo z naslednjo rodovno funkcijo:

$$G(x) = \frac{1}{1-x} \times \frac{1}{1-x^2}$$

Če izračunamo to funkcijo, dobimo rezultat 7 različnih kombinacij.

2.2 5. naloga

Rodovna funkcija za trikotniška števila je naslednja:

$$G(x) = \sum_{n=1}^{\infty} \frac{n(n+1)}{2} x^n$$

Stoto trikotniško število nato izračunamo sledeče:

$$T_{100} = \frac{100(100+1)}{2} = 5050$$

2.3 6. naloga

Vsako pozitivno celo število lahko zapišemo v binarnem številskem sestavu, kar pomeni, da lahko število zapišemo na natanko en način kot vsoto potenc števila 2. Rodovna funkcija za množico potenc števila 2 je sledeča:

$$G(x) = (1+x)(1+x^2)(1+x^4)(1+x^8)\dots$$

Vsak faktor $(1+x^k)$ v tej funkciji predstavlja ali je 2^k vključena v vrednost ali ne.

Ko razširimo $G(x)$, vsak faktor predstavlja seštevek določenih potenc števila 2. Ker je binarna predstavitev vsakega pozitivnega celega števila unikatna, lahko trdimo, da se vsako pozitivno celo število v razširitvi pojavi natanko enkrat.

Ker ima vsako pozitivno celo število svojo unikatno binarno predstavitev, in vsak faktor v razširitvi predstavlja unikatni seštevek potenc števila 2, lahko trdimo, da lahko vsako pozitivno celo število zapišemo na natanko en način kot vsoto različnih potenc števila 2.

3 Turingovi stroji

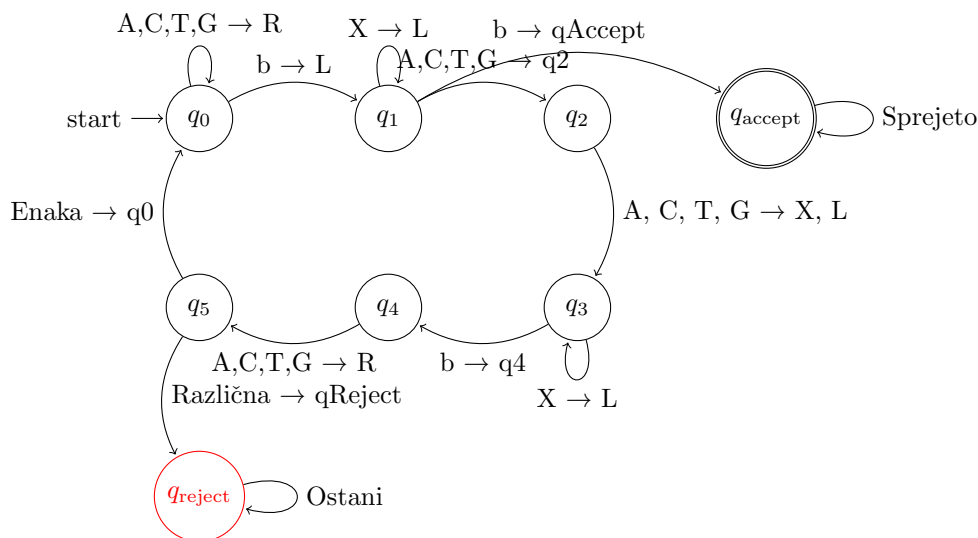
3.1 7. naloga

Turingov stroj za preverjanje, ali je beseda na traku palindrom, je sestavljen iz več delov. Najprej stroj poišče zadnji znak besede, nato preveri, ali so znaki na začetku in koncu enaki, in postopek ponavlja, dokler ne označi vseh znakov ali najde neskladje.

V prvem delu se glava stroja premika desno, dokler ne najde praznega znaka b, kar pomeni, da je dosegla konec besede. Nato se premakne levo, da najde zadnji znak.

V drugem delu stroj označi prvi neoznačen znak na začetku besede z X in se nato premakne na zadnji neoznačen znak. Tam preveri, ali je enak prvemu. Če sta enaka, se postopek ponovi: stroj premika glavo nazaj na začetek, označi naslednji neoznačen znak in nadaljuje preverjanje.

Ko stroj ne najde več neoznačenih znakov, se pomakne na začetek traku. Če ni našel neskladja, sprejme vhod in se ustavi. Če je kdaj našel dva različna znaka, zavrne vhod.



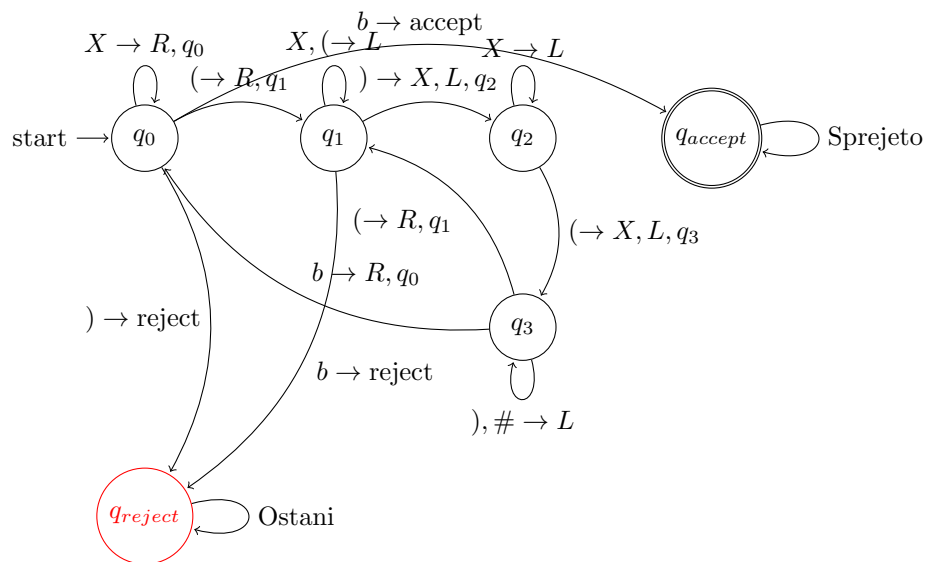
3.2 8. naloga

Turingov stroj za preverjanje, ali so oklepaji na traku pravilno vgnezdjeni, je sestavljen iz več delov. Najprej stroj poišče prvi zaklepaj $)$, nato preveri, ali ima ustrezen začetni oklepaj $($, in postopek ponavlja, dokler ne označi vseh ujemajočih se oklepajev.

V prvem delu se glava stroja premika desno in išče prvi $)$. Če ga najde, preide v iskanje njegovega ustreznega $($. Če pri iskanju prvega $)$ naleti na prazen simbol b , so vsi oklepaji že obdelani, in stroj sprejme vhod.

V drugem delu, ko stroj najde prvi $)$, ga označi z X in se začne premikati levo, dokler ne najde prvega $($, ki ga prav tako označi z X . Nato se znova premakne levo, dokler ne pride do začetka ali drugega znaka $($, kjer se preusmeri nazaj na iskanje naslednjega $)$ v desno smer.

Ko stroj ne najde več neobdelanih oklepajev, se pomakne na začetek traku. Če naleti na prazen simbol b , so vsi oklepaji pravilno ugnezdjeni in stroj sprejme vhod. Če kdaj naleti na neujemajoč se oklepaj ali ne najde para, stroj zavrne vhod.



3.3 9. naloga

Turingov stroj za preverjanje, ali dana beseda sestoji iz dveh enakih pod-besed, je sestavljen iz 2 delov. Prvi del poišče sredino dane besede, drugi del pa preveri, ali beseda sestoji iz dveh enakih podbesed.

V prvem delu turingov stroj paroma označuje črke na traku z leve in desne strani (1 zamenja z Y, 0 zamenja z 0). Ko označi zadnjo črko z desne proti levi, je glava stroja na začetku druge besede (beseda 101101 postane YXYYXY).

V drugem delu stroj najprej zamenja prvo podbesedo nazaj v 0 in 1 (101YXY). Nato, stroj zaporedoma označuje črke v levi podbesedi in briše črke iz desne podbesede (Y01bXY). Ko stroj ne najde več nobenih 0 ali 1, izvajanje zaključi.

